JOHANNES KEPLER
UNIVERSITY LINZ

Synthesis Optimizations and Low-Power Design



Simon Dorrer

KV VLSI Design, 336.051, WiSe 26/27

INSTITUTE FOR
INTEGRATED CIRCUITS &
QUANTUM COMPUTING

JOHANNES KEPLER
UNIVERSITÄT LINZ
Altenberger Straße 69
4040 Linz, Österreich
jku.at

Today's Goals

After this lecture, you can

- tell what **synthesis optimizes**, and what remains your job
- compare **ripple-carry** and **parallel-prefix** adders
- explain **resource sharing**, and check whether it happened
- shorten the critical path with a **pipeline**, explain **retiming**
- name the parts of the **power** of a CMOS circuit, and the levers
- estimate power with **OpenSTA**
- explain **clock gating** and insert it with Yosys



Chapter 7

Lecture notes: Chapter 7 of iic-jku.github.io/vlsi-design

What Synthesis Optimizes



Three Levels

- **Architecture**: algorithm, parallelism, latency, pipeline, memories
  - the designer: synthesis keeps the registers, a serial design stays serial
- **Operators** (Yosys): constants, unused logic, FSMs, widths, sharing
  - `opt`, `fsm`, `wreduce`, `alumacc`, `share`, then `techmap` builds the gates
- **Gates** (ABC): restructuring, mapping, sizing, buffering
  - only the Boolean function counts, but ABC starts from what Yosys hands over
- No synthesis tool turns a bubble sort into a quicksort

Switches in Yosys and LibreLane

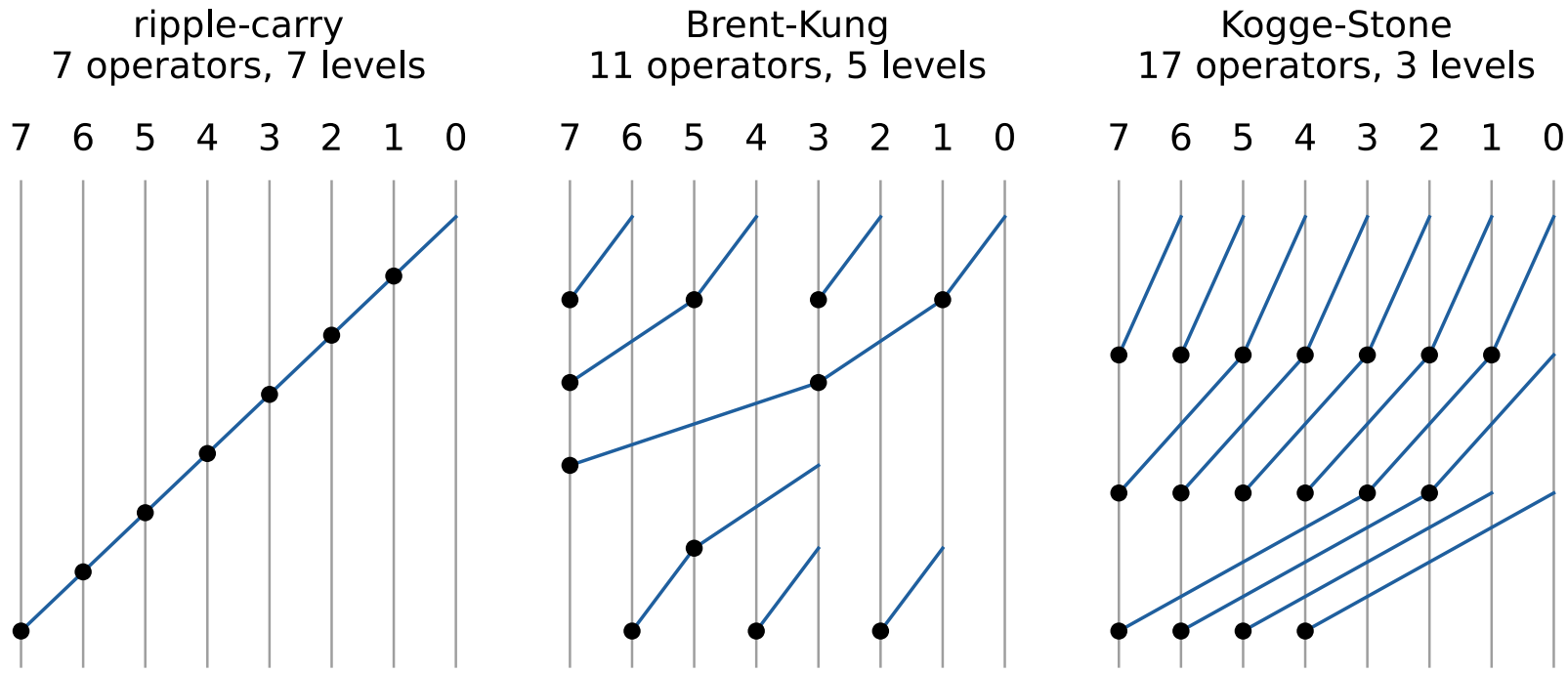
Optimization	Yosys	LibreLane
adder architecture	synth -extra-map	none
carry-save adder trees	always, synth -arith_tree	SYNTH_ARITH_TREE
Booth multipliers	synth -booth	SYNTH_MUL_BOOTH
resource sharing	share, opt_share	always on
mapping, sizing, buffering	abc and its options	SYNTH_STRATEGY
retiming	abc -dff	SYNTH_ABC_DFF
clock gating	clockgate	SYNTH_CLOCKGATE_...

- LibreLane: `AREA 0` by default, the template counter uses `DELAY 4`

Adders



Ripple-Carry and Prefix Adders



- $g_i = a_i\, b_i$, $p_i = a_i \oplus b_i$, $c_{i+1} = g_i + p_i\, c_i$
- $(G_h, P_h) \circ (G_l, P_l) = (G_h + P_h\, G_l, P_h\, P_l)$ is associative: a **tree** instead of a chain

Prefix Trees in Yosys

Adder	Levels	Operators	32 bits
ripple-carry	$n - 1$	$n - 1$	31 levels, 31 operators
Brent-Kung	$2 \log_2 n - 1$	$2n - 2 - \log_2 n$	9 levels, 57 operators
Kogge-Stone	$\log_2 n$	$n \log_2 n - n + 1$	5 levels, 129 operators

- Yosys: + becomes `$alu`, its carries a **Brent-Kung** tree by default
  - `synth -extra-map +/choices/kogge-stone.v`, also `sklansky.v`, `han-carlson.v`
- Many operands, multipliers: **carry-save trees**, **Booth** encoding

Exercise (Section 7.3): ripple or prefix, and does the adder architecture survive synthesis?

Resource Sharing



One Multiplier Instead of Two

```
p_o <= sel_q ? a_q * b_q : c_q * d_q;
```

`mul_sel`: 35,237 µm², 5.25 ns

```
p_o <= x * y;
assign x = sel_q ? a_q : c_q;
assign y = sel_q ? b_q : d_q;
```

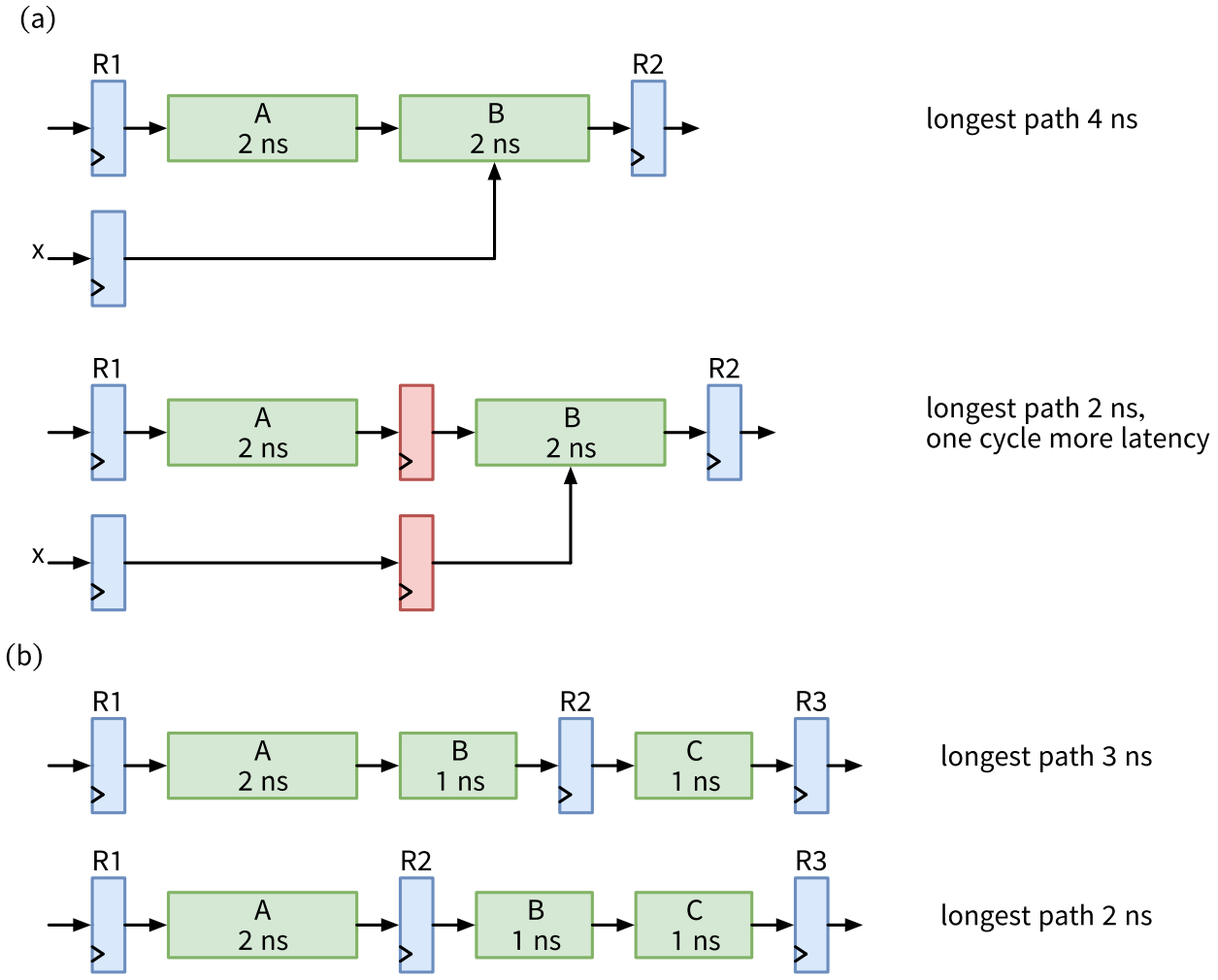
`mul_sel_shared`: 20,282 µm², 5.38 ns

- 16-bit operands: **42 % less area**, the multiplexers add 0.13 ns
- Yosys 0.70 did **not** share: `alumacc` makes `$macc_v2` cells, `share` skips them
  - with `synth -noalumacc`: shared, 20,251 µm²
- Write the sharing into the RTL where it matters, check with `stat`

Pipelining and Retiming



Pipeline Registers



- k stages: $t_{logic,max}/k$ per stage
- Throughput up, **latency** $+(k-1)$
- Overhead $t_{cq} + t_{su} + t_{unc} \approx$ **0.43 ns**
- Signals that belong together: **same number of registers**

Retiming in the Open Flow

- Retiming moves registers across logic, the outputs stay the same
  - changes register count and reset values, breaks the register matching of LEC (Chapter 5)
- Yosys passes registers to ABC only with `abc -dff`, LibreLane `SYNTH_ABC_DFF` (off)
  - only flip-flops **without reset**, before `dfflibmap`, then `dretime`
- This course: flip-flops with reset, **pipeline registers in the RTL**

Exercise (Section 7.5): pipeline the 32-bit adder in two stages: how many flip-flops, how fast?

Power Consumption



Where the Power Goes

- **Switching**: $P_{sw} = ½ \, a \, C \, V^2 \, f$, a = transitions per cycle, clock: a = 2
- **Internal**: inner nodes and short-circuit current, Liberty `internal_power`
  - a flip-flop at **every clock edge**, even if its value stays
- **Leakage**: transistors that are off, as long as the supply is on
- Levers: **V** squared, **f**, **C**, power gating against leakage
- Activity **a**: clock gating, operand isolation, fewer glitches

Power Analysis with OpenSTA

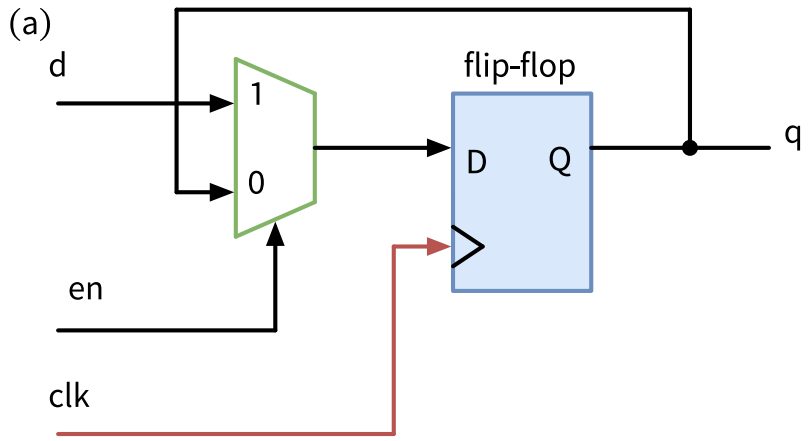
bs_reg, 1.2 V, 250 MHz	Internal	Switching	Leakage	Total
71 flip-flops	1.126 mW	0.030 mW	0.04 µW	1.156 mW
logic	0.329 mW	0.234 mW	0.15 µW	0.563 mW
total	1.455 mW	0.264 mW	0.19 µW	1.719 mW

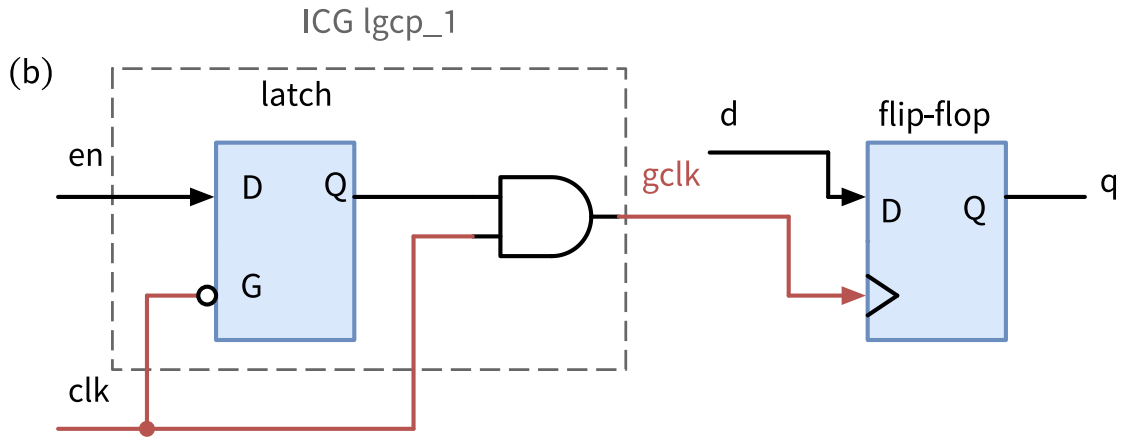
- Activity from `read_vcd` of a simulation, or the default: 0.1 at every input
- **67 %** in the flip-flops, 16.3 µW each: the clock reaches them in every cycle
- No clock buffers before the layout: the analysis misses the **clock tree**
- At 1.5 V: 2.78 mW (×1.62), in return 1.17 ns instead of 1.69 ns

Clock Gating



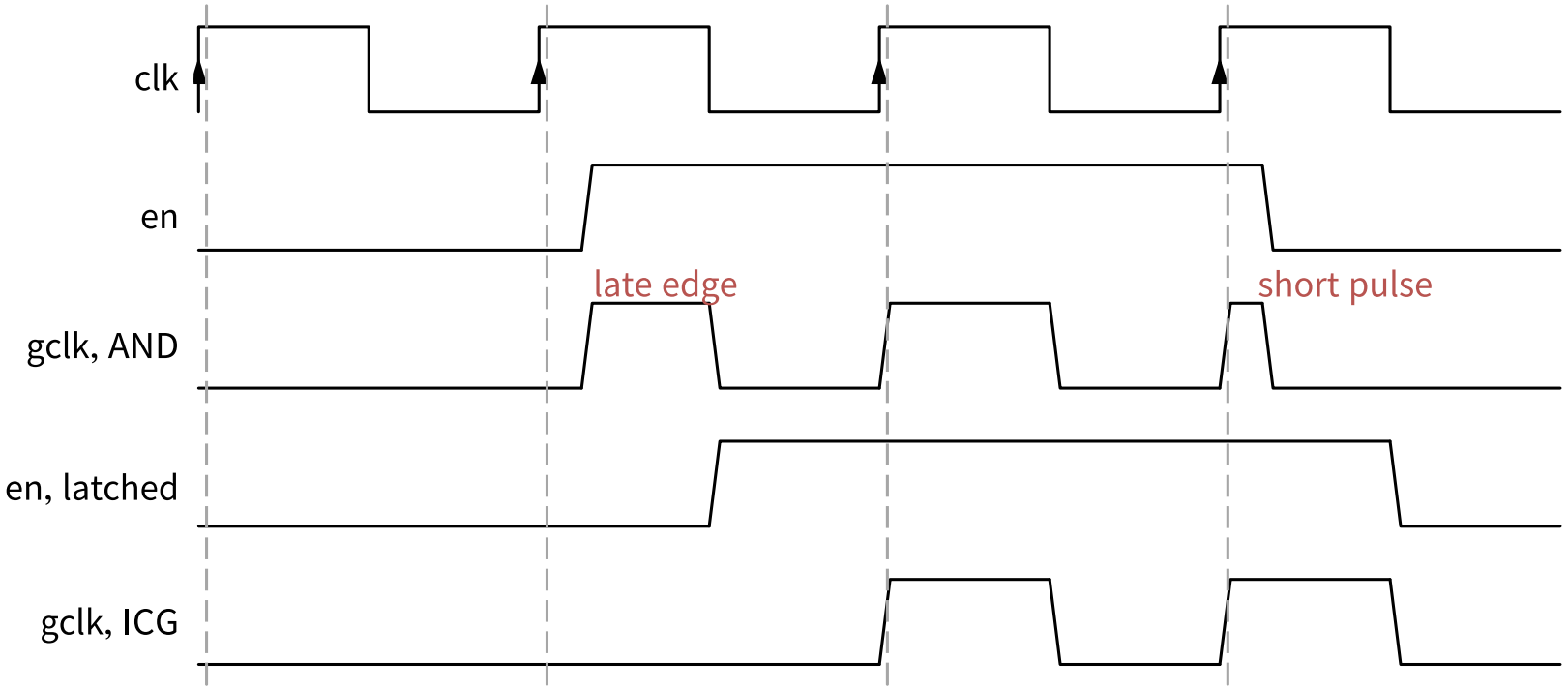
Enable: Multiplexer or Clock Gate





- SG13CMOS5L has **no flip-flop with enable**
  - (a) a multiplexer per bit, clocked in every cycle
- **Clock gating**: one gate for the whole register
  - (b) no multiplexers, no clock edges while en is low

Why an AND Gate Is Not Enough



- `en` changes after the edge, **while `clk` is high**
- ICG (integrated clock gate) `lgcp_1`: a latch, transparent while `clk` is **low**, then AND

Clock Gating in Yosys

```
clockgate -pos sg13cmos5l_lgcp_1 GATE:CLK:GCLK -min_net_size 4
```

- Between `synth` and `dfflibmap`: one ICG per group with the same clock and enable
- STA: clock through `CLK` to `GCLK`, a **clock gating check** at `GATE`
- `slgcp_1`: with scan enable `SCE`, Chapter 9
- LibreLane: `SYNTH_CLOCKGATE_MIN_WIDTH`, `..._POSEDGE_ICG`, off in SG13CMOS5L

Exercise (Section 7.7): gate the clock of a 32-bit register, and measure its power.

Summary

- Synthesis optimizes operators and gates, **not the architecture**
- Prefix adders: carries in a **tree**, Yosys builds Brent-Kung by default
- **Sharing** saves area for a multiplexer delay, write it into the RTL
- **Pipelining**: throughput up, latency and registers up
- Power: switching, internal, leakage, **V^2** and **activity** are the levers
- Before clock tree synthesis, power analysis misses the **clock tree**
- **Clock gating** needs an ICG with a latch, Yosys `clockgate`

Exercises

1. Ripple or Prefix? (7.3)
2. Pipeline the Adder (7.5)
3. Gate the Clock (7.7)

All code in `content/synth_opt/code` of the course repository, every number reproduced by `scripts/check_synth_opt.sh`



Chapter 7



JOHANNES KEPLER
UNIVERSITY LINZ